



DPG SCHOOL OF TECHNOLOGY & MANAGEMENT

(A unit of DPG Degree College, Sec-34, Gurugram)

(Affiliated to MDU Rohtak)

Recognized 2(f) by UGC & Accredited with 'A' Grade by NAAC



A
PRACTICAL FILE
OF
COMPUTER GRAPHICS
(24BCA404DS04)

BCA – 303

IN PARTIAL FULLFILLMENT OF THE REQUIREMENT OF BACHELOR OF
COMPUTER APPLICATION



SUBMITTED TO
MS SHIKHA MATHUR
ASSISSTANT PROFESSOR
DEPARTMENT OF COMPUTER
SCIENCE
DPG STM

SUBMITTED BY
AKSHAT PRASAD
S/O SANJAY PRASAD
STUDENT ID - 145609
BCA – 4A

SUBMITTED ON MONDAY , 20TH APRIL , 2026

MDU ROHTAK , DPG UNIVERSE

INDEX

#	PROGRAM DESCRIPTION	PAGE NO.	SIGN
1	Implement a program to define and use basic graphics primitives such as points, lines, and polygons.		
2	Implement a program to draw a line using the Digital Differential Analyzer (DDA) algorithm.		
3	Create a program to draw a line using Bresenham's line drawing algorithm.		
4	Write a program to draw a circle using the Midpoint Circle algorithm.		
5	Implement a program to draw a circle using Bresenham's circle drawing algorithm.		
6	Create a program to perform 2D transformations: translation, rotation, and scaling on a given set of points.		
7	Write a program to perform mirror reflection of a 2D shape across the x-axis and y-axis.		
8	Create a program to draw basic two-dimensional objects like rectangles, triangles, and polygons using inbuilt functions.		
9	Write a program to implement the Cohen-Sutherland line clipping algorithm to clip a line within a rectangular clipping window.		
10	Write a program to apply a shearing transformation to a 2D object (like a square or triangle) and display the original and transformed objects.		
11	Write a program to apply various coloring techniques to 2D pictures.		
12	Write a program to implement line clipping using the Cohen-Sutherland line clipping algorithm.		
13	Write a program to perform 3D transformations: translation, rotation, and scaling on a set of 3D points.		
14	Implement a program to demonstrate simple animations using transformations such as translation, rotation, and scaling.		
15	Write a program to apply various coloring techniques to 3D object.		

Program 1 :- Implement a program to define and use basic graphics primitives such as points, lines, and polygons.

Code –

```
#include <graphics.h>
#include <conio.h>
int main()
{
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "C:\\Turboc3\\BGI");

    // Drawing a Point
    putpixel(100, 100, WHITE);

    // Drawing a Line
    line(150, 100, 300, 100);

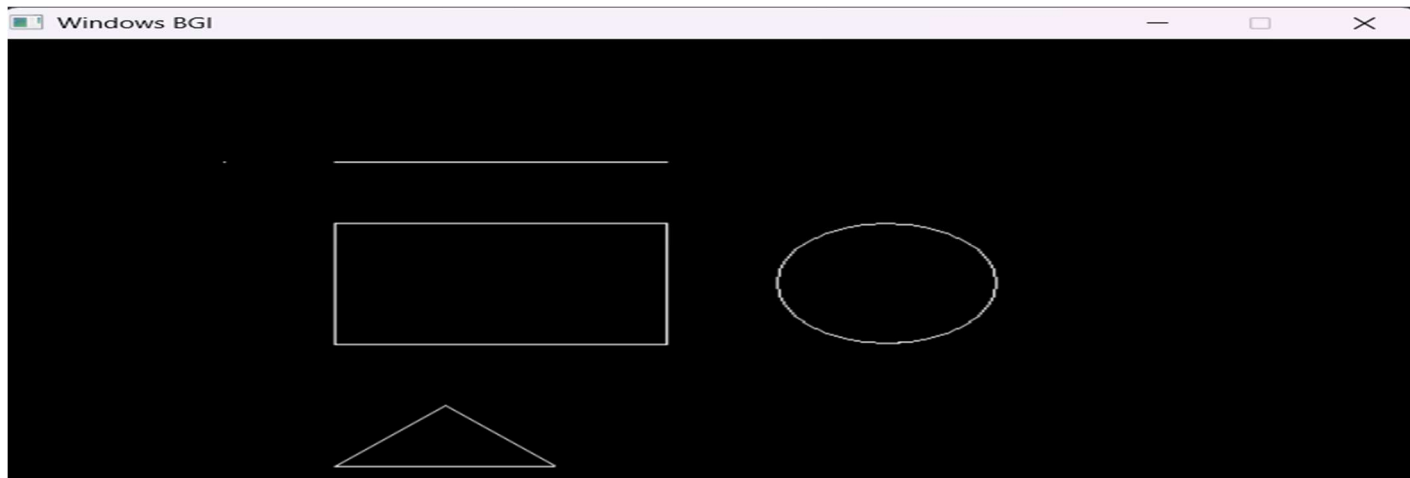
    // Drawing a Rectangle
    rectangle(150, 150, 300, 250);

    // Drawing a Circle
    circle(400, 200, 50);

    // Drawing a Polygon (Triangle)
    int points[] = {200, 300, 250, 350, 150, 350, 200, 300};
    drawpoly(4, points);

    getch();
    closegraph();
    return 0;
}
```

Expected Output –



Program 2 :- Implement a program to draw a line using the Digital Differential Analyzer (DDA) algorithm.

Code :-

```
#include <graphics.h>
#include <math.h>

int main()
{
    int gd = DETECT, gm;
    initgraph(&gd, &gm, (char*)"");

    // Fixed coordinates
    int x1 = 100, y1 = 100;
    int x2 = 400, y2 = 300;

    int dx = x2 - x1;
    int dy = y2 - y1;

    int steps;

    if (abs(dx) > abs(dy))
        steps = abs(dx);
    else
        steps = abs(dy);

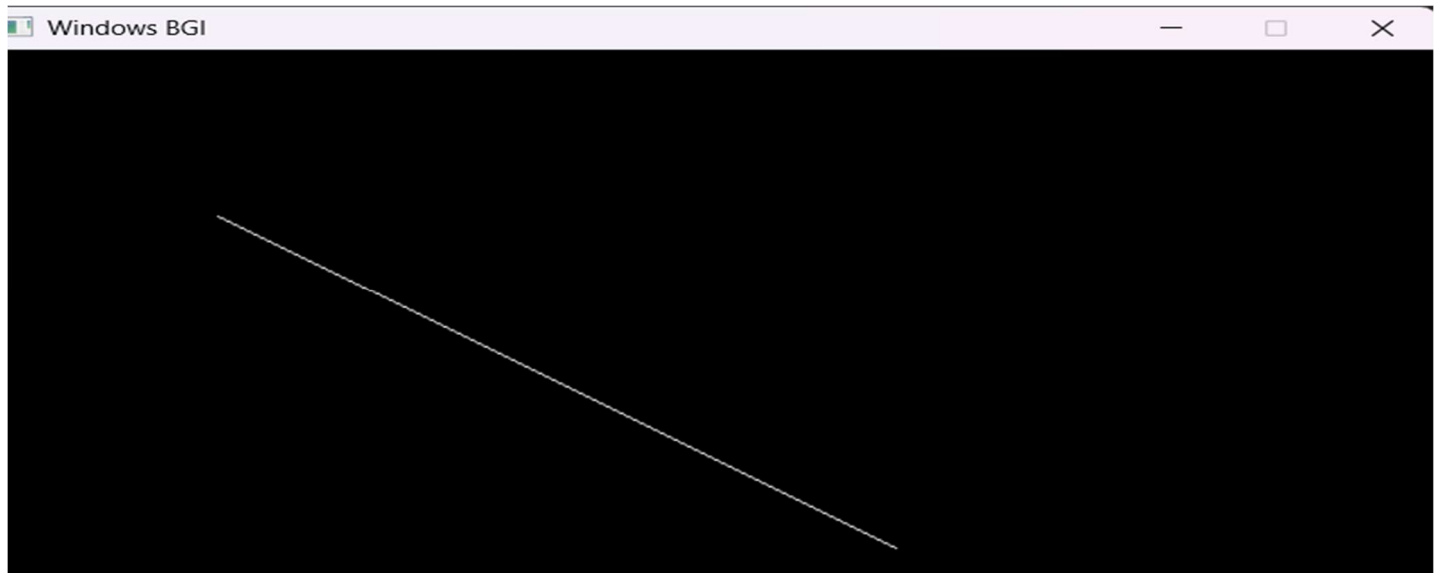
    float xIncrement = dx / (float)steps;
    float yIncrement = dy / (float)steps;

    float x = x1;
    float y = y1;

    for (int i = 0; i <= steps; i++)
    {
        putpixel(round(x), round(y), WHITE);
        x += xIncrement;
        y += yIncrement;
    }
}
```

```
getch();  
closegraph();  
return 0;  
}
```

Expected Output :-



Program 3 :- Create a program to draw a line using Bresenham's line drawing algorithm.

Code –

```
#include <graphics.h>
#include <math.h>
#include <iostream>
#include <cstdlib>

void bresenham(int x1, int y1, int x2, int y2)
{
    int dx = abs(x2 - x1);
    int dy = abs(y2 - y1);
    int sx = (x1 < x2) ? 1 : -1;
    int sy = (y1 < y2) ? 1 : -1;
    int err = dx - dy;

    while (true)
    {
        putpixel(x1, y1, WHITE);

        if (x1 == x2 && y1 == y2)
            break;

        int err2 = err * 2;

        if (err2 > -dy)
        {
            err -= dy;
            x1 += sx;
        }

        if (err2 < dx)
        {
            err += dx;
            y1 += sy;
        }
    }
}

int main()
{
```

```

int gd = DETECT, gm;
initgraph(&gd, &gm, (char*)"");

int x1, y1, x2, y2;

std::cout << "Enter the coordinates of the first point (x1 y1): ";
std::cin >> x1 >> y1;

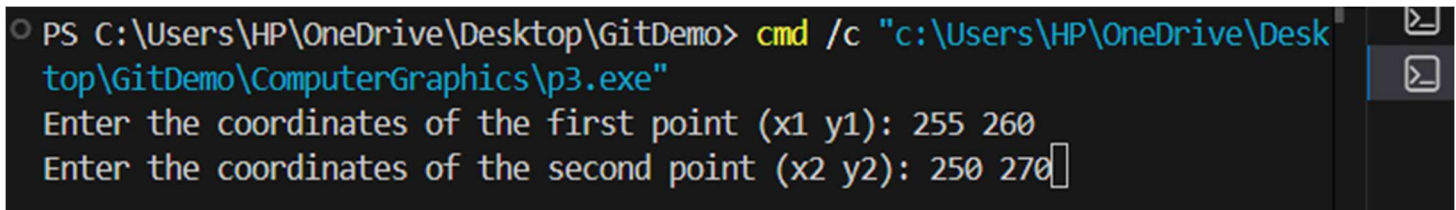
std::cout << "Enter the coordinates of the second point (x2 y2): ";
std::cin >> x2 >> y2;

bresenham(x1, y1, x2, y2);

getch();
closegraph();
return 0;
}

```

I/O Screen :-

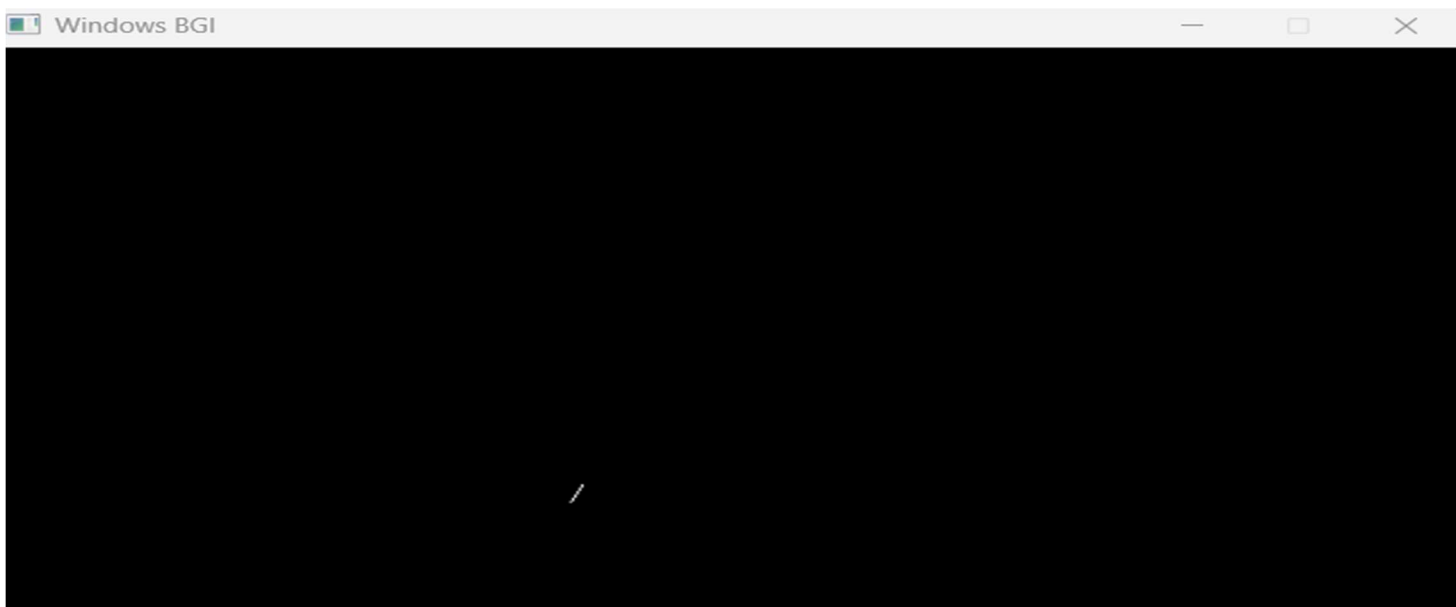


```

PS C:\Users\HP\OneDrive\Desktop\GitDemo> cmd /c "c:\Users\HP\OneDrive\Desktop\GitDemo\ComputerGraphics\p3.exe"
Enter the coordinates of the first point (x1 y1): 255 260
Enter the coordinates of the second point (x2 y2): 250 270

```

Expected Output :-



Program 4 :- Write a program to draw a circle using the Midpoint Circle algorithm.

Code –

```
#include <graphics.h>
#include <iostream>
#include <cmath>
#include <cstdlib>
#include <vector>

class Point {
public:
    int x, y;
    Point(int x, int y) : x(x), y(y) {}
};

void drawCircle(int xc, int yc, int r) {
    int x = 0;
    int y = r;
    int d = 3 - 2 * r;

    while (x <= y) {
        // Draw the eight symmetrical points
        putpixel(xc + x, yc + y, WHITE);
        putpixel(xc - x, yc + y, WHITE);
        putpixel(xc + x, yc - y, WHITE);
        putpixel(xc - x, yc - y, WHITE);
        putpixel(xc + y, yc + x, WHITE);
        putpixel(xc - y, yc + x, WHITE);
        putpixel(xc + y, yc - x, WHITE);
        putpixel(xc - y, yc - x, WHITE);

        if (d < 0) {
            d = d + 4 * x + 6;
        } else {
            d = d + 4 * (x - y) + 10;
            y--;
        }
        x++;
    }
}
```

```

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, (char*)"");

    int xc, yc, r;

    std::cout << "Enter the center coordinates of the circle (xc yc): ";
    std::cin >> xc >> yc;

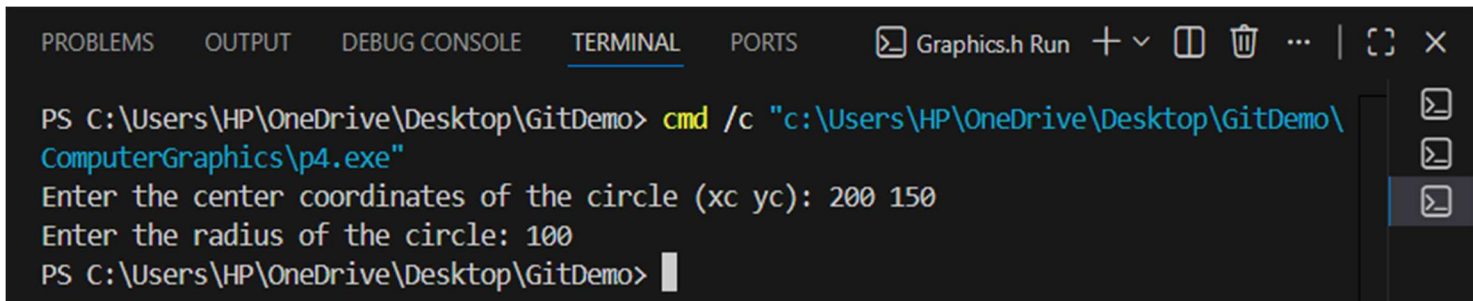
    std::cout << "Enter the radius of the circle: ";
    std::cin >> r;

    drawCircle(xc, yc, r);

    getch();
    closegraph();
    return 0;
}

```

I/O Screen :-

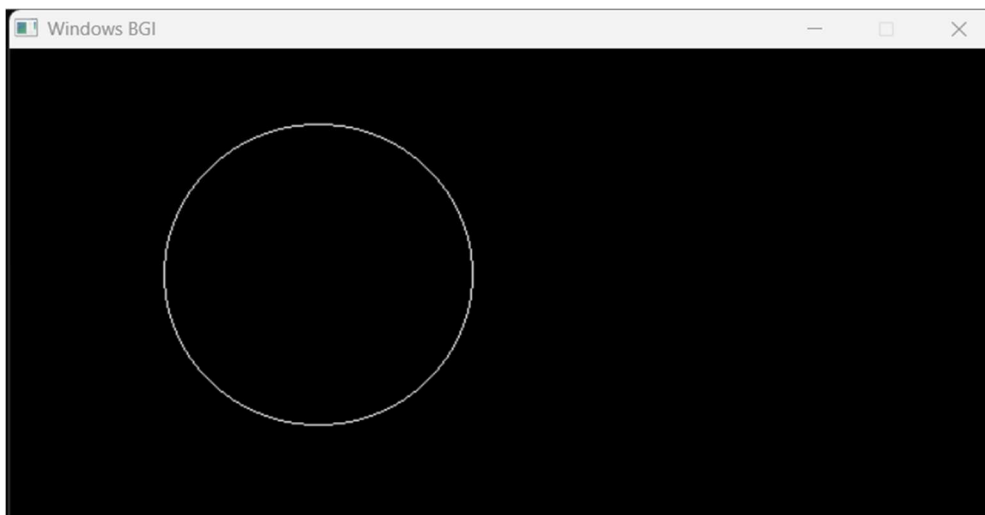


```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Graphics.h Run + - [ ] [X] ... | [ ] [X] [X]
PS C:\Users\HP\OneDrive\Desktop\GitDemo> cmd /c "c:\Users\HP\OneDrive\Desktop\GitDemo\
ComputerGraphics\p4.exe"
Enter the center coordinates of the circle (xc yc): 200 150
Enter the radius of the circle: 100
PS C:\Users\HP\OneDrive\Desktop\GitDemo>

```

Expected Output :-



Program 5 :- Implement a program to draw a circle using Bresenham's circle drawing algorithm.

Code –

```
#include <graphics.h>
#include <iostream>
#include <cmath>

void drawCircle(int xc, int yc, int r) {
    int x = 0;
    int y = r;
    int d = 3 - 2 * r;

    while (x <= y) {
        // Draw the eight symmetrical points
        putpixel(xc + x, yc + y, RED);
        putpixel(xc - x, yc + y, RED);
        putpixel(xc + x, yc - y, RED);
        putpixel(xc - x, yc - y, RED);
        putpixel(xc + y, yc + x, RED);
        putpixel(xc - y, yc + x, RED);
        putpixel(xc + y, yc - x, RED);
        putpixel(xc - y, yc - x, RED);

        if (d < 0) {
            d = d + 4 * x + 6;
        } else {
            d = d + 4 * (x - y) + 10;
            y--;
        }
        x++;
    }
}

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, (char*)"");

    int xc, yc, r;

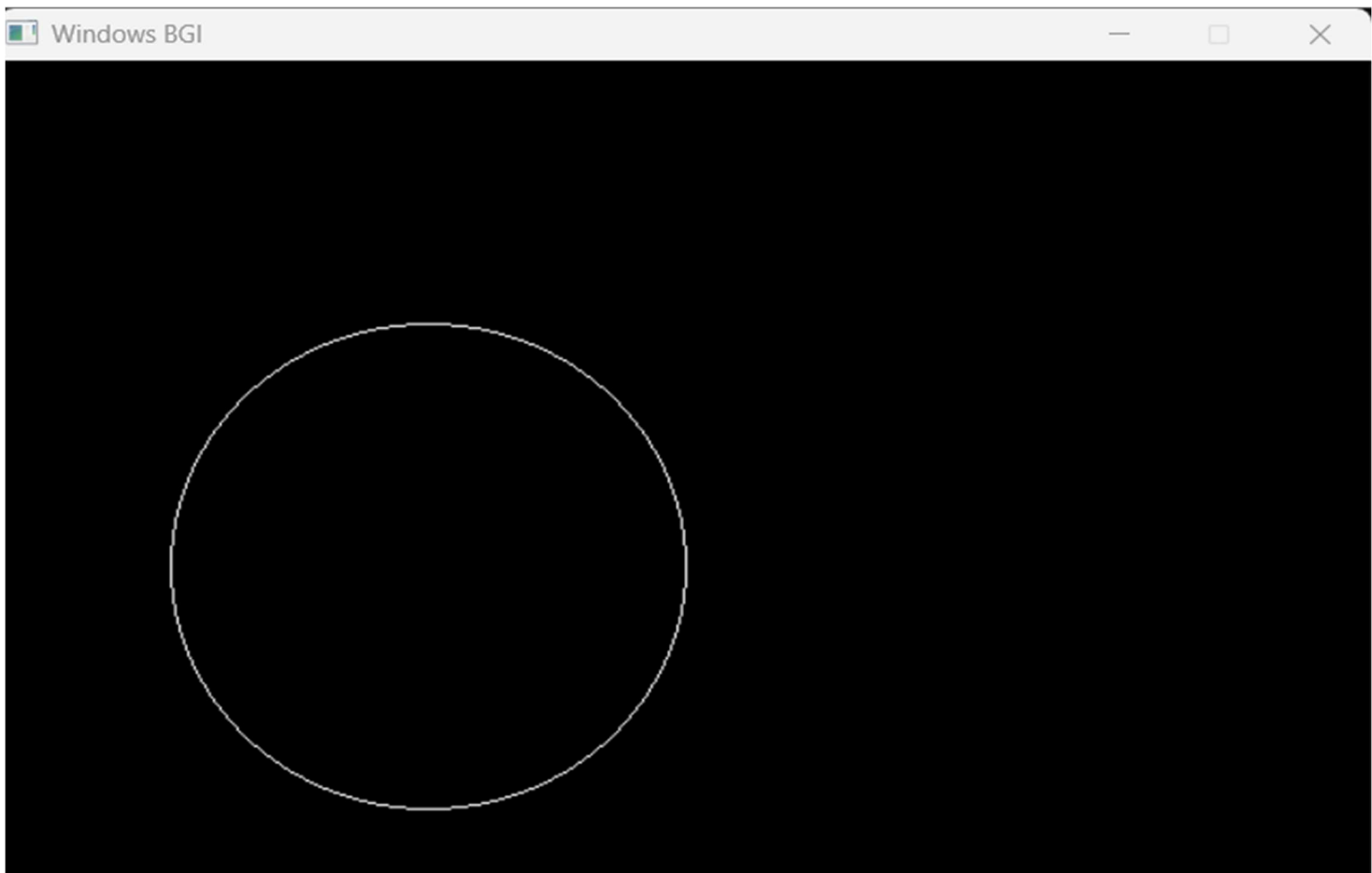
    std::cout << "Enter the center coordinates of the circle (xc yc): ";
    std::cin >> xc >> yc;
```

```
std::cout << "Enter the radius of the circle: ";  
std::cin >> r;  
  
drawCircle(xc, yc, r);  
  
getch();  
closegraph();  
return 0;  
}
```

I/O Screen :-

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Graphics.h Run + - [ ] [X] ... | [ ] [X] [X]  
● PS C:\Users\HP\OneDrive\Desktop\GitDemo> cmd /c "c:\Users\HP\OneDrive\Desktop\GitDemo\ComputerGraphics\p5.exe"  
Enter the center coordinates of the circle (xc yc): 200 250  
Enter the radius of the circle: 120  
○ PS C:\Users\HP\OneDrive\Desktop\GitDemo> |
```

Expected Output :-



Program 6 :- Create a program to perform 2D transformations: translation, rotation, and scaling on a given set of points.

Code –

```
#include <iostream>
#include <vector>
#include <cmath>
using namespace std;

struct Point {
    double x, y;
};

void translate(vector<Point>& points, double tx, double ty) {
    for (auto& point : points) {
        point.x += tx;
        point.y += ty;
    }
}

void rotate(vector<Point>& points, double angle) {
    double radians = angle * M_PI / 180.0;
    double cosA = cos(radians);
    double sinA = sin(radians);

    for (auto& point : points) {
        double xNew = point.x * cosA - point.y * sinA;
        double yNew = point.x * sinA + point.y * cosA;
        point.x = xNew;
        point.y = yNew;
    }
}

void scale(vector<Point>& points, double sx, double sy) {
    for (auto& point : points) {
        point.x *= sx;
        point.y *= sy;
    }
}

void printPoints(const vector<Point>& points) {
    for (const auto& point : points) {
```

```

        cout << "(" << point.x << ", " << point.y << ")" << endl;
    }
}

int main() {
    vector<Point> points = {{1, 2}, {3, 4}, {5, 6}};

    cout << "Original Points:" << endl;
    printPoints(points);

    translate(points, 2, 3);
    cout << "\nAfter Translation (tx=2, ty=3):" << endl;
    printPoints(points);

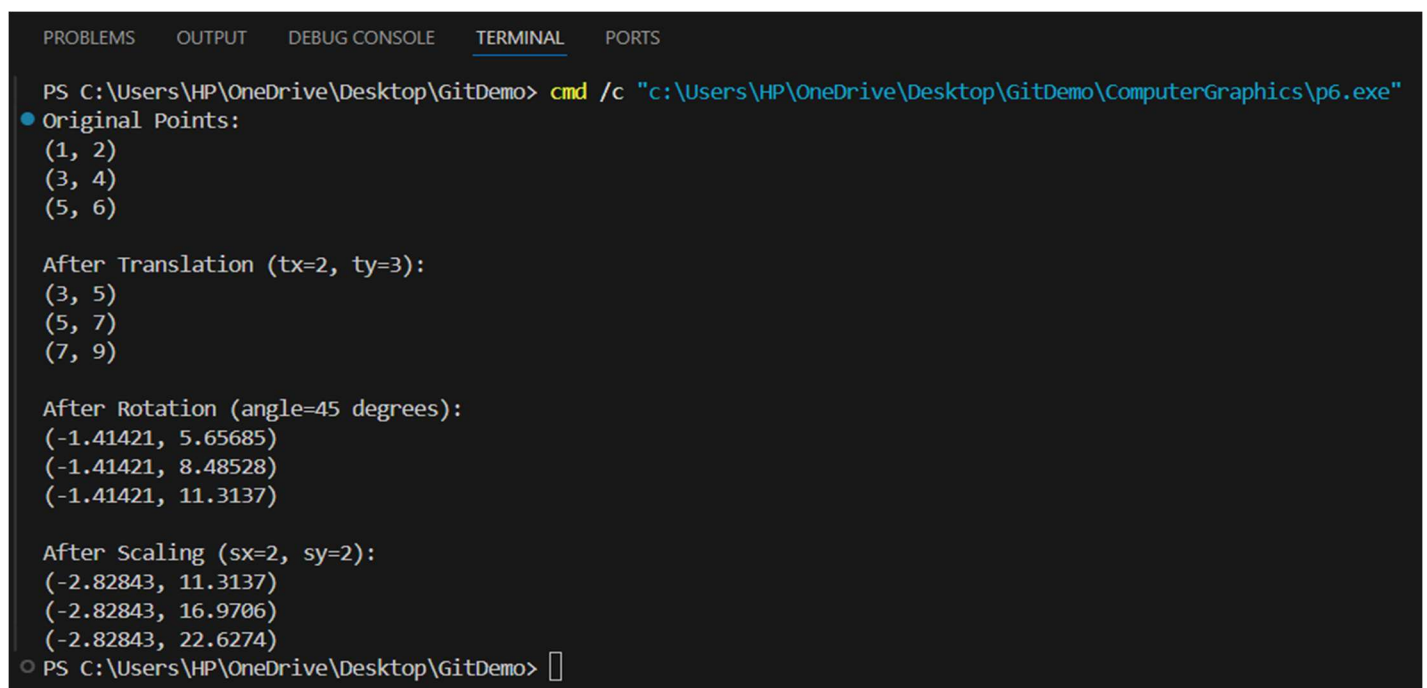
    rotate(points, 45);
    cout << "\nAfter Rotation (angle=45 degrees):" << endl;
    printPoints(points);

    scale(points, 2, 2);
    cout << "\nAfter Scaling (sx=2, sy=2):" << endl;
    printPoints(points);

    return 0;
}

```

Terminal Output :-



The screenshot shows a terminal window with the following output:

```

PS C:\Users\HP\OneDrive\Desktop\GitDemo> cmd /c "c:\Users\HP\OneDrive\Desktop\GitDemo\ComputerGraphics\p6.exe"
● Original Points:
(1, 2)
(3, 4)
(5, 6)

After Translation (tx=2, ty=3):
(3, 5)
(5, 7)
(7, 9)

After Rotation (angle=45 degrees):
(-1.41421, 5.65685)
(-1.41421, 8.48528)
(-1.41421, 11.3137)

After Scaling (sx=2, sy=2):
(-2.82843, 11.3137)
(-2.82843, 16.9706)
(-2.82843, 22.6274)
PS C:\Users\HP\OneDrive\Desktop\GitDemo>

```

Program 7 :- Write a program to perform mirror reflection of a 2D shape across the x-axis and y-axis.

Code –

```
#include <graphics.h>
#include <iostream>
#include <cmath>
#include <cstdlib>
#include <vector>

using namespace std;

// Structure for Point
struct Point {
    int x, y;
};

// Function to draw triangle
void drawTriangle(vector<Point> &shape, int color) {
    setcolor(color);

    for (int i = 0; i < shape.size(); i++) {
        int next = (i + 1) % shape.size();
        line(shape[i].x, shape[i].y,
            shape[next].x, shape[next].y);
    }
}

// Mirror across X-axis
vector<Point> mirrorX(vector<Point> shape, int midY) {
    vector<Point> reflected;

    for (auto p : shape) {
        Point temp;
        temp.x = p.x;
        temp.y = 2 * midY - p.y; // Reflection formula
        reflected.push_back(temp);
    }

    return reflected;
}
```

```

// Mirror across Y-axis
vector<Point> mirrorY(vector<Point> shape, int midX) {
    vector<Point> reflected;

    for (auto p : shape) {
        Point temp;
        temp.x = 2 * midX - p.x; // Reflection formula
        temp.y = p.y;
        reflected.push_back(temp);
    }

    return reflected;
}

int main() {

    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");

    int midX = getmaxx() / 2;
    int midY = getmaxy() / 2;

    // Draw X and Y axis
    setcolor(WHITE);
    line(0, midY, getmaxx(), midY); // X-axis
    line(midX, 0, midX, getmaxy()); // Y-axis

    // Original Triangle
    vector<Point> triangle;

    triangle.push_back({midX, midY - 100});
    triangle.push_back({midX - 100, midY + 100});
    triangle.push_back({midX + 100, midY + 100});

    // Draw original (RED)
    drawTriangle(triangle, RED);

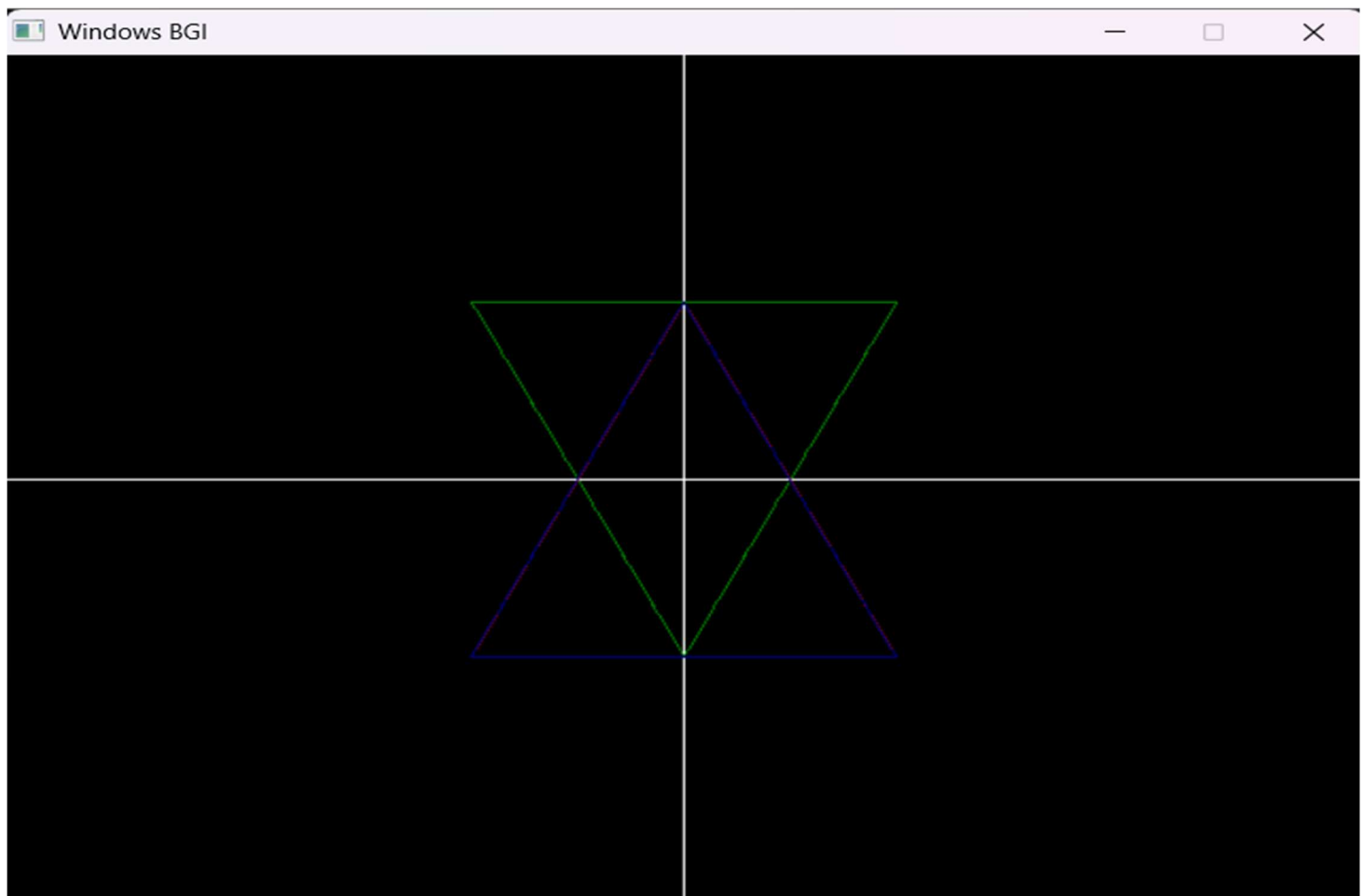
    // Reflection across X-axis (GREEN)
    vector<Point> reflectX = mirrorX(triangle, midY);
    drawTriangle(reflectX, GREEN);

    // Reflection across Y-axis (BLUE)

```

```
vector<Point> reflectY = mirrorY(triangle, midX);  
drawTriangle(reflectY, BLUE);  
  
getch();  
closegraph();  
  
return 0;  
}
```

Expected Output :-



Program 8 :- Create a program to draw basic two-dimensional objects like rectangles, triangles, and polygons using inbuilt functions.

Code –

```
#include <graphics.h>
#include <conio.h>
#include <iostream>
#include <cmath>
using namespace std;

// function to draw a rectangle
void drawRectangle(int x1, int y1, int x2, int y2) {
    rectangle(x1, y1, x2, y2);
}

// function to draw a triangle
void drawTriangle(int x1, int y1, int x2, int y2, int x3, int y3) {
    line(x1, y1, x2, y2);
    line(x2, y2, x3, y3);
    line(x3, y3, x1, y1);
}

// function to draw a polygon
void drawPolygon(int points[][2], int n) {
    for (int i = 0; i < n; i++) {
        line(points[i][0], points[i][1], points[(i + 1) % n][0], points[(i + 1) % n][1]);
    }
}

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "C:\\\\Turboc3\\\\BGI");

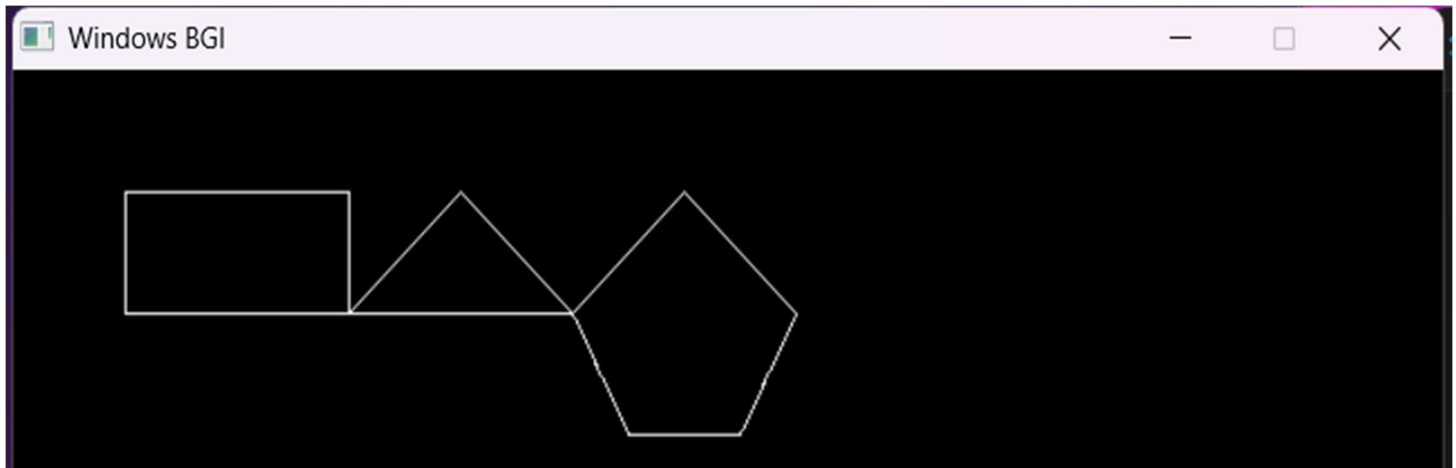
    // Draw a rectangle
    drawRectangle(50, 50, 150, 100);

    // Draw a triangle
    drawTriangle(200, 50, 250, 100, 150, 100);

    // Draw a polygon (pentagon)
    int points[5][2] = {{300, 50}, {350, 100}, {325, 150}, {275, 150}, {250, 100}};
    drawPolygon(points, 5);
}
```

```
    getch();  
    closegraph();  
    return 0;  
}
```

Expected Output :-



Program 9 :- Write a program to implement the Cohen-Sutherland line clipping algorithm to clip a line within a rectangular clipping window.

Code –

```
#include <graphics.h>
#include <iostream>
using namespace std;

const int INSIDE = 0;
const int LEFT = 1;
const int RIGHT = 2;
const int BOTTOM = 4;
const int TOP = 8;

int xmin = 100, ymin = 100, xmax = 300, ymax = 300;

int computeCode(double x, double y)
{
    int code = INSIDE;

    if (x < xmin)
        code |= LEFT;
    else if (x > xmax)
        code |= RIGHT;

    if (y < ymin)
        code |= BOTTOM;
    else if (y > ymax)
        code |= TOP;

    return code;
}

void cohenSutherlandClip(double x1, double y1, double x2, double y2)
{
    int code1 = computeCode(x1, y1);
    int code2 = computeCode(x2, y2);

    bool accept = false;

    while (true)
```

```

{
    if ((code1 == 0) && (code2 == 0))
    {
        accept = true;
        break;
    }
    else if (code1 & code2)
    {
        break;
    }
    else
    {
        int code_out;
        double x, y;

        if (code1 != 0)
            code_out = code1;
        else
            code_out = code2;

        if (code_out & TOP)
        {
            x = x1 + (x2 - x1) * (ymax - y1) / (y2 - y1);
            y = ymax;
        }
        else if (code_out & BOTTOM)
        {
            x = x1 + (x2 - x1) * (ymin - y1) / (y2 - y1);
            y = ymin;
        }
        else if (code_out & RIGHT)
        {
            y = y1 + (y2 - y1) * (xmax - x1) / (x2 - x1);
            x = xmax;
        }
        else if (code_out & LEFT)
        {
            y = y1 + (y2 - y1) * (xmin - x1) / (x2 - x1);
            x = xmin;
        }

        if (code_out == code1)
        {

```

```

        x1 = x;
        y1 = y;
        code1 = computeCode(x1, y1);
    }
    else
    {
        x2 = x;
        y2 = y;
        code2 = computeCode(x2, y2);
    }
}

if (accept)
{
    setcolor(RED);
    line(x1, y1, x2, y2);
}
}

int main()
{
    int gd = DETECT, gm;

    initgraph(&gd, &gm, (char*)"");

    rectangle(xmin, ymin, xmax, ymax);

    double x1 = 50, y1 = 50, x2 = 350, y2 = 350;

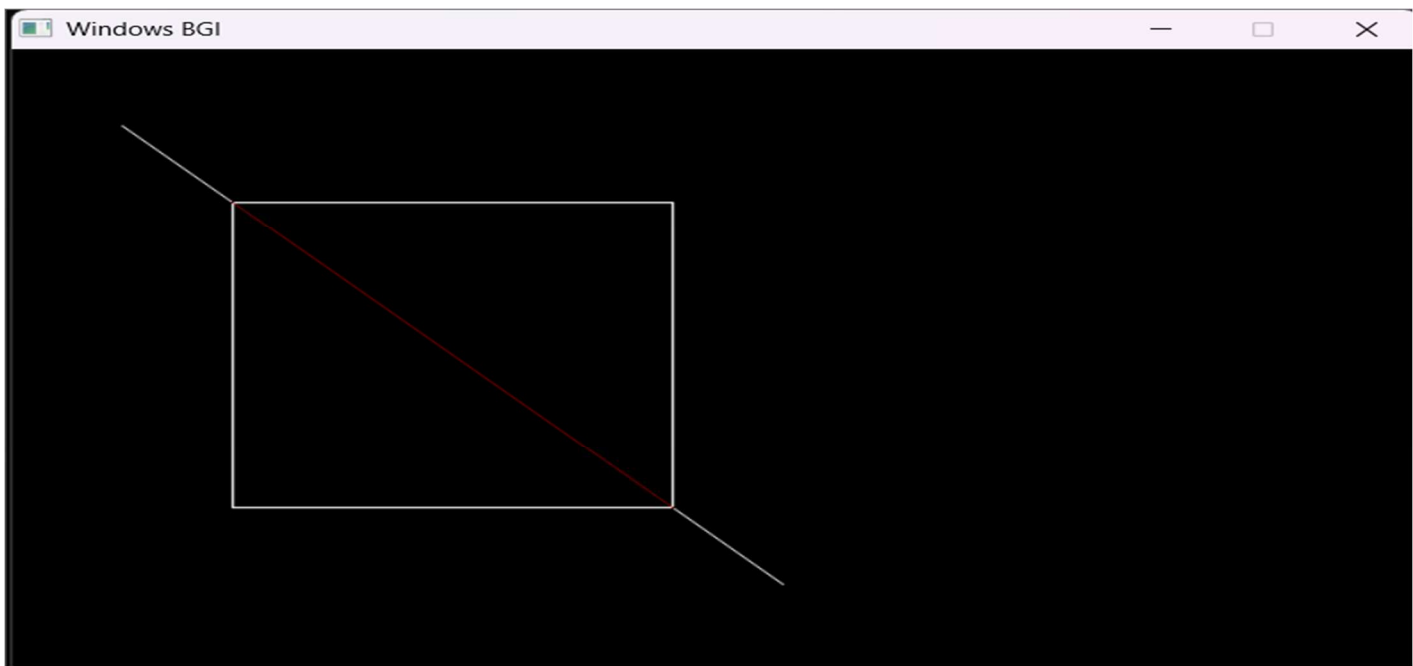
    setcolor(WHITE);
    line(x1, y1, x2, y2);

    cohenSutherlandClip(x1, y1, x2, y2);

    getch();
    closegraph();
}

```

Expected Output :-



Program 10 :- Write a program to apply a shearing transformation to a 2D object (like a square or triangle) and display the original and transformed objects.

Code –

```
#include <graphics.h>
#include <iostream>
using namespace std;

const int INSIDE = 0;
const int LEFT = 1;
const int RIGHT = 2;
const int BOTTOM = 4;
const int TOP = 8;

int xmin = 100, ymin = 100, xmax = 300, ymax = 300;

int computeCode(double x, double y)
{
    int code = INSIDE;

    if (x < xmin)
        code |= LEFT;
    else if (x > xmax)
        code |= RIGHT;

    if (y < ymin)
        code |= BOTTOM;
    else if (y > ymax)
        code |= TOP;

    return code;
}

void cohenSutherlandClip(double x1, double y1, double x2, double y2)
{
    int code1 = computeCode(x1, y1);
    int code2 = computeCode(x2, y2);

    bool accept = false;

    while (true)
```

```

{
    if ((code1 == 0) && (code2 == 0))
    {
        accept = true;
        break;
    }
    else if (code1 & code2)
    {
        break;
    }
    else
    {
        int code_out;
        double x, y;

        if (code1 != 0)
            code_out = code1;
        else
            code_out = code2;

        if (code_out & TOP)
        {
            x = x1 + (x2 - x1) * (ymax - y1) / (y2 - y1);
            y = ymax;
        }
        else if (code_out & BOTTOM)
        {
            x = x1 + (x2 - x1) * (ymin - y1) / (y2 - y1);
            y = ymin;
        }
        else if (code_out & RIGHT)
        {
            y = y1 + (y2 - y1) * (xmax - x1) / (x2 - x1);
            x = xmax;
        }
        else if (code_out & LEFT)
        {
            y = y1 + (y2 - y1) * (xmin - x1) / (x2 - x1);
            x = xmin;
        }

        if (code_out == code1)
        {

```

```

        x1 = x;
        y1 = y;
        code1 = computeCode(x1, y1);
    }
    else
    {
        x2 = x;
        y2 = y;
        code2 = computeCode(x2, y2);
    }
}

if (accept)
{
    setcolor(RED);
    line(x1, y1, x2, y2);
}
}

int main()
{
    int gd = DETECT, gm;

    initgraph(&gd, &gm, (char*)"");

    rectangle(xmin, ymin, xmax, ymax);

    double x1 = 50, y1 = 50, x2 = 350, y2 = 350;

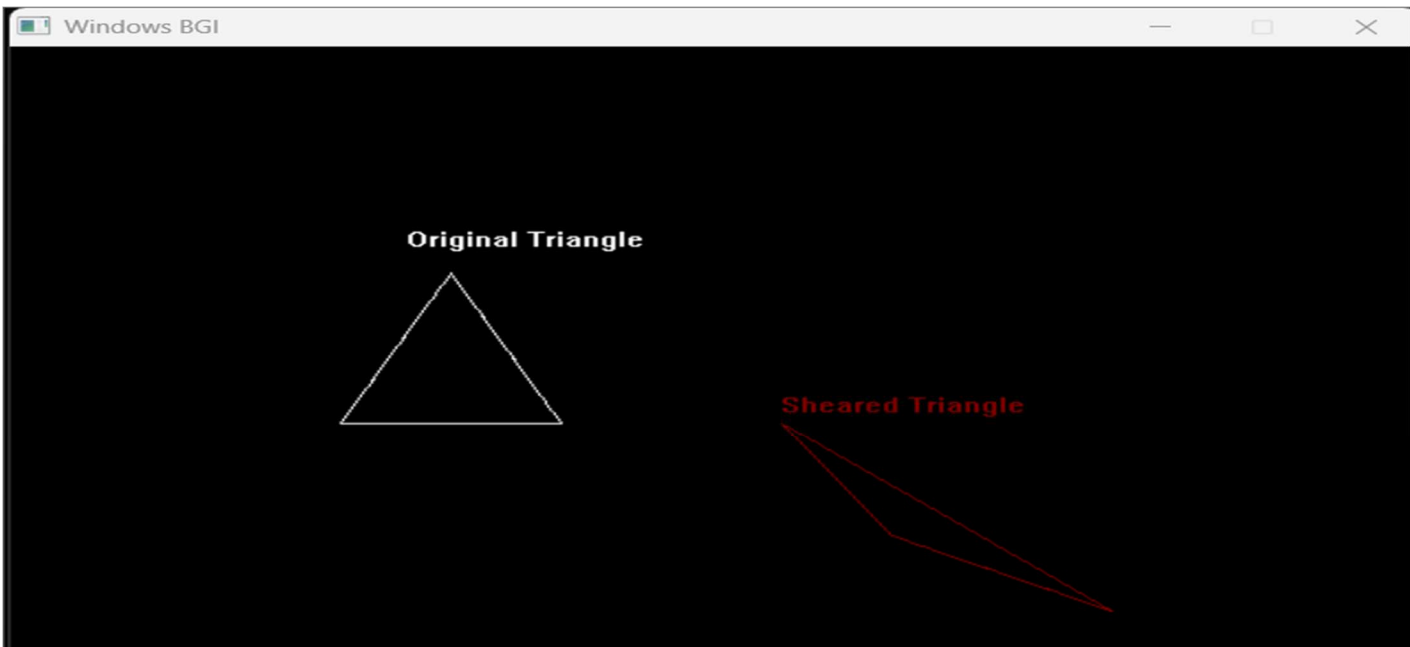
    setcolor(WHITE);
    line(x1, y1, x2, y2);

    cohenSutherlandClip(x1, y1, x2, y2);

    getch();
    closegraph();
}

```

Expected Output :-



Program 11 :- Write a program to apply various coloring techniques to 2D pictures.

Code –

```
#include <graphics.h>
#include <iostream>
using namespace std;

int main()
{
    int gd = DETECT, gm;
    initgraph(&gd, &gm, (char*)"");

    // SKY
    setfillstyle(SOLID_FILL, LIGHTBLUE);
    bar(0,0,640,250);

    // GROUND
    setfillstyle(SOLID_FILL, GREEN);
    bar(0,250,640,480);

    // SUN
    setcolor(YELLOW);
    circle(500,80,40);
    setfillstyle(SOLID_FILL,YELLOW);
    floodfill(500,80,YELLOW);

    // HOUSE BODY
    setcolor(WHITE);
    rectangle(200,200,350,320);
    setfillstyle(SOLID_FILL,RED);
    floodfill(210,210,WHITE);

    // ROOF
    setcolor(WHITE);
    line(200,200,275,150);
    line(275,150,350,200);
    setfillstyle(HATCH_FILL,BROWN);
    floodfill(275,180,WHITE);

    // DOOR
    setcolor(WHITE);
```

```

rectangle(260,250,300,320);
setfillstyle(SOLID_FILL,BROWN);
floodfill(270,260,WHITE);

// WINDOW
setcolor(WHITE);
rectangle(215,230,245,260);
setfillstyle(SOLID_FILL,CYAN);
floodfill(220,235,WHITE);

// TREE TRUNK
setcolor(WHITE);
rectangle(120,230,140,320);
setfillstyle(SOLID_FILL,BROWN);
floodfill(125,240,WHITE);

// TREE LEAVES
setcolor(GREEN);
circle(130,200,35);
setfillstyle(SOLID_FILL,GREEN);
floodfill(130,200,GREEN);

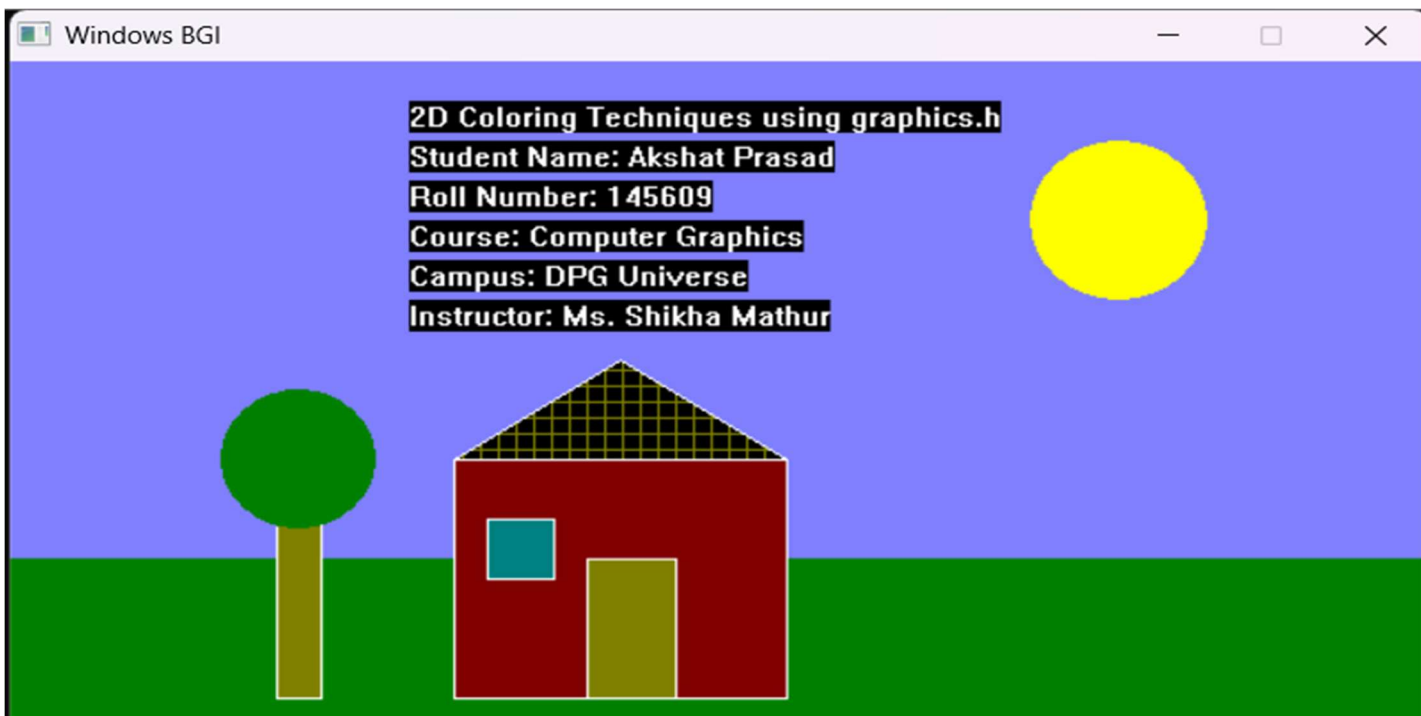
// TEXT TITLE
setcolor(WHITE);
outtextxy(180,20,"2D Coloring Techniques using graphics.h");

// TEXT CREDITS SHOWING STUDENT NAME AND ROLL NUMBER
outtextxy(180,40,"Student Name: Akshat Prasad");
outtextxy(180,60,"Roll Number: 145609");
outtextxy(180,80,"Course: Computer Graphics");
outtextxy(180,100,"Campus: DPG Universe");
outtextxy(180,120,"Instructor: Ms. Shikha Mathur");

getch();
closegraph();
}

```

Expected Output :-



Program 12 :- Write a program to implement line clipping using the Cohen-Sutherland line clipping algorithm.

Code –

```
#include <graphics.h>
#include <iostream>
using namespace std;

const int INSIDE = 0;
const int LEFT = 1;
const int RIGHT = 2;
const int BOTTOM = 4;
const int TOP = 8;

int xmin = 150, ymin = 100, xmax = 450, ymax = 350;

int computeCode(int x, int y)
{
    int code = INSIDE;

    if(x < xmin)
        code |= LEFT;
    else if(x > xmax)
        code |= RIGHT;

    if(y < ymin)
        code |= BOTTOM;
    else if(y > ymax)
        code |= TOP;

    return code;
}

void cohenSutherland(int x1,int y1,int x2,int y2)
{
    int code1 = computeCode(x1,y1);
    int code2 = computeCode(x2,y2);

    bool accept = false;

    while(true)
```

```

{
    if(code1 == 0 && code2 == 0)
    {
        accept = true;
        break;
    }
    else if(code1 & code2)
    {
        break;
    }
    else
    {
        int code_out;
        int x,y;

        if(code1 != 0)
            code_out = code1;
        else
            code_out = code2;

        if(code_out & TOP)
        {
            x = x1 + (x2-x1)*(ymax-y1)/(y2-y1);
            y = ymax;
        }
        else if(code_out & BOTTOM)
        {
            x = x1 + (x2-x1)*(ymin-y1)/(y2-y1);
            y = ymin;
        }
        else if(code_out & RIGHT)
        {
            y = y1 + (y2-y1)*(xmax-x1)/(x2-x1);
            x = xmax;
        }
        else
        {
            y = y1 + (y2-y1)*(xmin-x1)/(x2-x1);
            x = xmin;
        }

        if(code_out == code1)
        {

```

```

        x1 = x;
        y1 = y;
        code1 = computeCode(x1,y1);
    }
    else
    {
        x2 = x;
        y2 = y;
        code2 = computeCode(x2,y2);
    }
}

if(accept)
{
    setcolor(RED);
    line(x1,y1,x2,y2);
}
}

int main()
{
    int gd = DETECT, gm;

    initgraph(&gd,&gm,(char*)"");

    rectangle(xmin,ymin,xmax,ymax);

    int x1 = 100, y1 = 50, x2 = 500, y2 = 400;

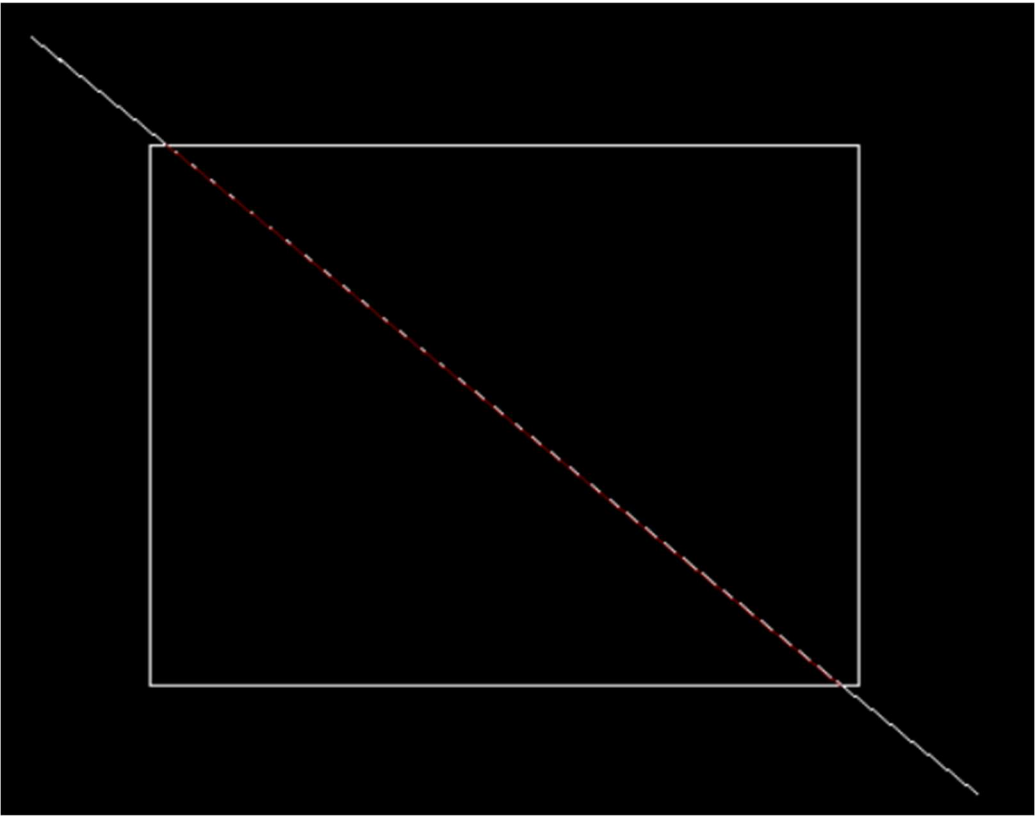
    setcolor(WHITE);
    line(x1,y1,x2,y2);

    cohenSutherland(x1,y1,x2,y2);

    getch();
    closegraph();
}

```

Expected Output :-



Program 13 :- Write a program to perform 3D transformations: translation, rotation, and scaling on a set of 3D points.

Code –

```
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
    int n;
    float x[20], y[20], z[20];

    cout << "Enter number of points: ";
    cin >> n;

    cout << "Enter coordinates (x y z):\n";

    for(int i = 0; i < n; i++)
    {
        cout << "Point " << i+1 << ": ";
        cin >> x[i] >> y[i] >> z[i];
    }

    int choice;

    cout << "\nChoose Transformation\n";
    cout << "1. Translation\n";
    cout << "2. Scaling\n";
    cout << "3. Rotation about Z-axis\n";
    cout << "Enter choice: ";
    cin >> choice;

    if(choice == 1)
    {
        float tx, ty, tz;

        cout << "Enter translation factors (tx ty tz): ";
        cin >> tx >> ty >> tz;

        cout << "\nTranslated Points:\n";
```

```

for(int i = 0; i < n; i++)
{
    cout << "("
        << x[i] + tx << ", "
        << y[i] + ty << ", "
        << z[i] + tz << ")\n";
}
}

else if(choice == 2)
{
    float sx, sy, sz;

    cout << "Enter scaling factors (sx sy sz): ";
    cin >> sx >> sy >> sz;

    cout << "\nScaled Points:\n";

    for(int i = 0; i < n; i++)
    {
        cout << "("
            << x[i] * sx << ", "
            << y[i] * sy << ", "
            << z[i] * sz << ")\n";
    }
}

else if(choice == 3)
{
    float angle;

    cout << "Enter rotation angle (degree): ";
    cin >> angle;

    float rad = angle * 3.14159265 / 180;

    cout << "\nRotated Points (about Z-axis):\n";

    for(int i = 0; i < n; i++)
    {
        float x1 = x[i] * cos(rad) - y[i] * sin(rad);
        float y1 = x[i] * sin(rad) + y[i] * cos(rad);
    }
}

```

```

        cout << "("
            << x1 << ", "
            << y1 << ", "
            << z[i] << ")\n";
    }
}

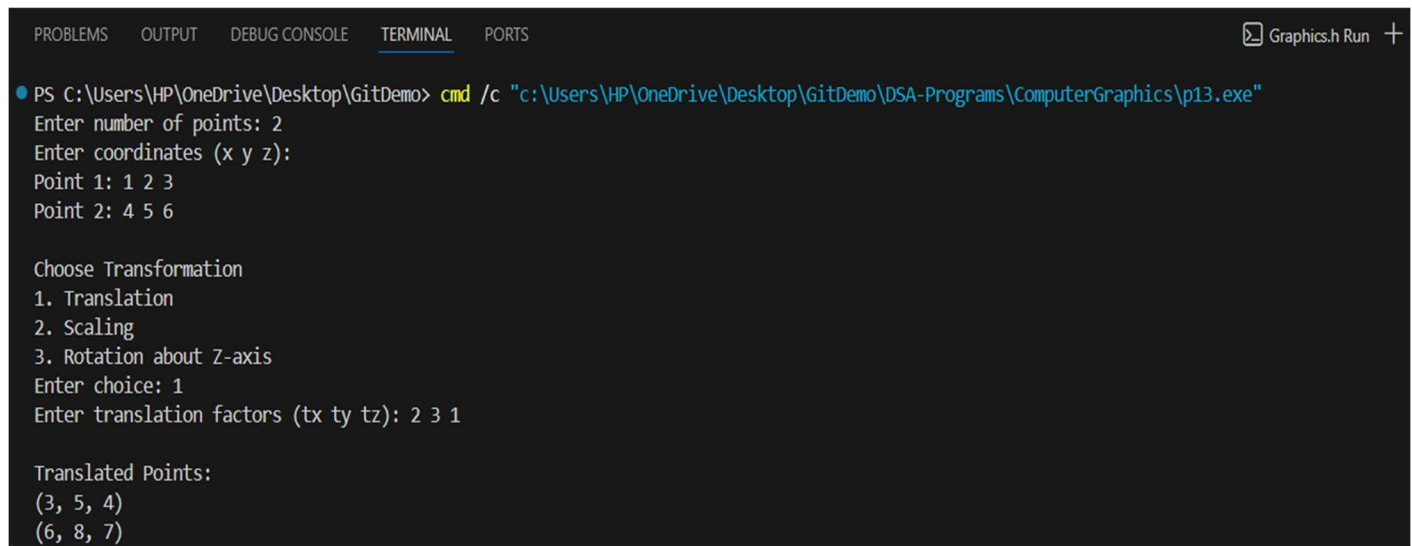
else
{
    cout << "Invalid choice\n";
}

return 0;
}

```

Expected Output Based on Input Scenario :-

Case 1 –



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Graphics.h Run +

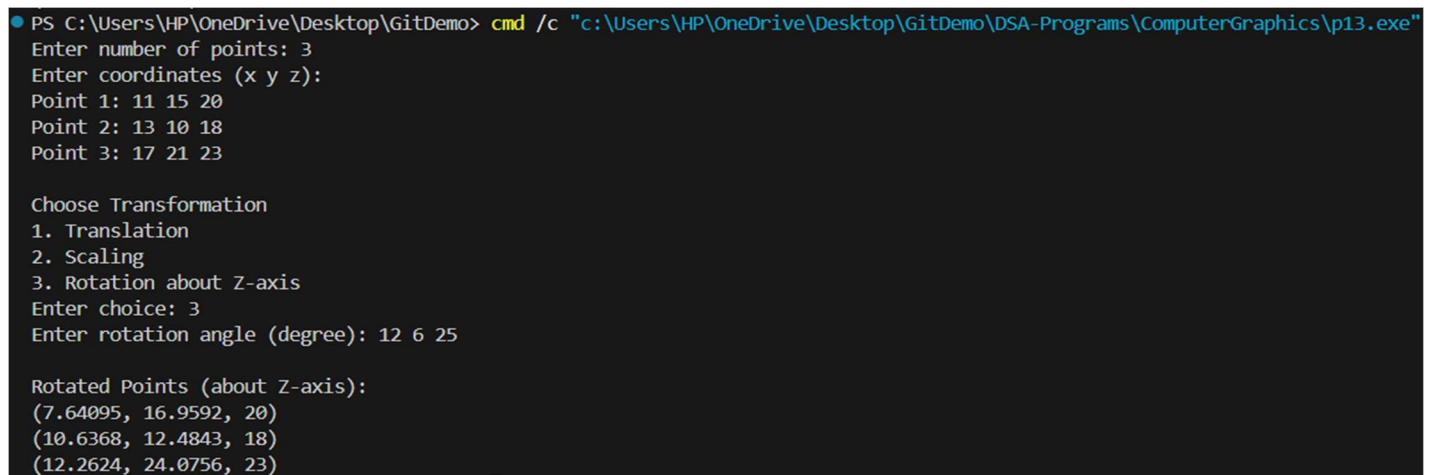
PS C:\Users\HP\OneDrive\Desktop\GitDemo> cmd /c "c:\Users\HP\OneDrive\Desktop\GitDemo\DSA-Programs\ComputerGraphics\p13.exe"
Enter number of points: 2
Enter coordinates (x y z):
Point 1: 1 2 3
Point 2: 4 5 6

Choose Transformation
1. Translation
2. Scaling
3. Rotation about Z-axis
Enter choice: 1
Enter translation factors (tx ty tz): 2 3 1

Translated Points:
(3, 5, 4)
(6, 8, 7)

```

Case 2 –



```

PS C:\Users\HP\OneDrive\Desktop\GitDemo> cmd /c "c:\Users\HP\OneDrive\Desktop\GitDemo\DSA-Programs\ComputerGraphics\p13.exe"
Enter number of points: 3
Enter coordinates (x y z):
Point 1: 11 15 20
Point 2: 13 10 18
Point 3: 17 21 23

Choose Transformation
1. Translation
2. Scaling
3. Rotation about Z-axis
Enter choice: 3
Enter rotation angle (degree): 12 6 25

Rotated Points (about Z-axis):
(7.64095, 16.9592, 20)
(10.6368, 12.4843, 18)
(12.2624, 24.0756, 23)

```

Case 3 –

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Graphics.h Run +
PS C:\Users\HP\OneDrive\Desktop\GitDemo> cmd /c "c:\Users\HP\OneDrive\Desktop\GitDemo\DSA-Programs\ComputerGraphics\p13.exe"
Enter number of points: 6
Enter coordinates (x y z):
Point 1: 1 2 3
Point 2: 4 5 6
Point 3: 7 8 9
Point 4: 10 11 12
Point 5: 13 14 15
Point 6: 16 17 18

Choose Transformation
1. Translation
2. Scaling
3. Rotation about Z-axis
Enter choice: 2
Enter scaling factors (sx sy sz): 10 12 11

Scaled Points:
(10, 24, 33)
(40, 60, 66)
(70, 96, 99)
(100, 132, 132)
(130, 168, 165)
(160, 204, 198)
```

Program 14 :- Implement a program to demonstrate simple animations using transformations such as translation, rotation, and scaling.

Code –

```
#include <graphics.h>
#include <math.h>
#include <dos.h>

void drawBranding()
{
    setcolor(YELLOW);
    settextstyle(BOLD_FONT,HORIZ_DIR,2);
    outtextxy(180,10,"COMPUTER GRAPHICS PRACTICAL");

    setcolor(WHITE);
    settextstyle(DEFAULT_FONT,HORIZ_DIR,1);

    outtextxy(20,40,"Student Name: Akshat Prasad");
    outtextxy(20,60,"Roll Number: 145609");
}

int main()
{
    int gd = DETECT, gm;
    initgraph(&gd,&gm,(char*)"");

    int x = 0;
    float angle = 0;
    int r = 25;

    while(!kbhit())
    {
        cleardevice();

        // SKY
        setfillstyle(SOLID_FILL,LIGHTBLUE);
        bar(0,140,640,240);

        // GROUND
        setfillstyle(SOLID_FILL,GREEN);
```

```

bar(0,240,640,480);

// SUN (SCALING)
setcolor(YELLOW);
setfillstyle(SOLID_FILL,YELLOW);
circle(550,170,r);

// ROAD
setcolor(WHITE);
line(0,320,640,320);

// CAR BODY (TRANSLATION)
setcolor(RED);
rectangle(x,270,x+120,310);

// CAR TOP
line(x+20,270,x+50,240);
line(x+50,240,x+90,240);
line(x+90,240,x+110,270);

// WHEELS
setcolor(BLACK);
circle(x+30,320,20);
circle(x+90,320,20);

// WHEEL ROTATION
setcolor(WHITE);
line(x+30,320,
     x+30+20*cos(angle),
     320+20*sin(angle));

line(x+90,320,
     x+90+20*cos(angle),
     320+20*sin(angle));

// DRAW BRANDING (ALWAYS VISIBLE)
drawBranding();

delay(40);

// Transformations
x += 5;
angle += 0.25;

```

```

r += 1;

if(r > 40)
    r = 25;

if(x > 640)
    x = -120;
}
closegraph();
}

```

Expected Output (Animation Running Output) :-

➤ For Demonstration 3 snapshots of Output Screen is Attach here .

➤ **Animation Area –**

- 🚗 Car moving → Translation
- 🌀 Wheel spokes rotating → Rotation
- ☀️ Sun expanding → Scaling

Snapshot (1 to 3) –





Program 15 :- Write a program to apply various coloring techniques to 3D object.

Code –

```
#include <graphics.h>
#include <dos.h>
#include <conio.h>

void drawCube()
{
    setcolor(WHITE);

    rectangle(80,180,140,240);
    rectangle(100,160,160,220);

    line(80,180,100,160);
    line(140,180,160,160);
    line(140,240,160,220);

    setfillstyle(SOLID_FILL,RED);
    floodfill(90,190,WHITE);

    outtextxy(95,250,"CUBE");
}

void drawCuboid()
{
    setcolor(WHITE);

    rectangle(220,180,300,240);
    rectangle(250,160,330,220);

    line(220,180,250,160);
    line(300,180,330,160);
    line(300,240,330,220);

    setfillstyle(HATCH_FILL,CYAN);
    floodfill(230,190,WHITE);

    outtextxy(255,250,"CUBOID");
}
```

```
void drawPyramid()
{
    setcolor(WHITE);

    line(420,240,460,170);
    line(460,170,500,240);
    line(420,240,500,240);

    line(460,170,460,240);

    setfillstyle(SLASH_FILL,YELLOW);
    floodfill(460,210,WHITE);

    outtextxy(440,250,"PYRAMID");
}

void drawSphere()
{
    setcolor(WHITE);

    circle(150,340,40);

    setfillstyle(SOLID_FILL,LIGHTBLUE);
    floodfill(150,340,WHITE);

    outtextxy(130,390,"SPHERE");
}

void drawCylinder()
{
    setcolor(WHITE);

    ellipse(350,310,0,360,40,15);
    line(310,310,310,360);
    line(390,310,390,360);
    ellipse(350,360,0,360,40,15);

    setfillstyle(WIDE_DOT_FILL,GREEN);
    floodfill(350,330,WHITE);

    outtextxy(330,390,"CYLINDER");
}
```

```

int main()
{
    int gd = DETECT, gm;

    initgraph(&gd,&gm,(char*)"");

    // Background
    setfillstyle(SOLID_FILL, DARKGRAY);
    bar(0,0,640,480);

    setcolor(WHITE);
    settxtstyle(BOLD_FONT,HORIZ_DIR,3);
    outtextxy(120,20,"3D OBJECT COLORING TECHNIQUES");

    // Draw objects
    drawCube();
    drawCuboid();
    drawPyramid();
    drawSphere();
    drawCylinder();

    getch();
    closegraph();
}

```

Expected Output :-

